THE ARCHITECT'S FIELD NOTES

BEHAVIORAL INTERVIEW STUDY GUIDE / 01 OF 02

AI Design #73 — Unified Enterprise Agentic Platform

THINK CLEARLY. DECIDE WITH EVIDENCE. DELIVER WITH CONFIDENCE.

Evidence-based STAR answers, senior-level judgment, and focused interview rehearsal.

PREPARED FOR SENIOR / PRINCIPAL SOLUTION ARCHITECT INTERVIEWS

READ · REFLECT · PRACTICE

01

YOUR STUDY COMPANION

Capstone: Enterprise IT Operations Assistant

`Stage 3 of 6 — Minimal Working Architecture`

NAVIGATION

Inside this guide

01	1. Minimal component architecture
02	2. Ownership by component
03	3. Define the API and typed tool contracts
04	4. Happy-path request sequence
05	5. Trust boundaries and authorization
06	6. Failure path — monitoring data unavailable
07	7. Baseline execution limits
08	8. What we will implement in Stage 4
09	Stage 3 summary

CHAPTER

1. Minimal component architecture

We will now turn the Stage 1 requirements and Stage 2 concepts into a concrete system design.

Our goal is to build the smallest architecture that can:

1. Accept an investigation request for `INC-42`.
2. Retrieve authorized metrics and deployment evidence.
3. Use an LLM to analyze the evidence.
4. Return findings and a safe recommendation.
5. Never modify production.

One bounded investigation component and two approved read-only tools. No MCP, A2A, RAG or distributed platform services yet.

1.1 Logical components

V1 — READ-ONLY INVESTIGATION ASSISTANT





Logical request/data flow, not a deployment topology. Tool responses pass back through the workflow before reaching the model.

Deployment decision: The API, workflow, tools, and validator can run in one Python application. They are not required to be separate services.

Why each component exists

Component	Responsibility	Boundary	Why we need it
Web UI	Accept request and show findings		Human interaction
Investigation API	Verify identity and enforce request access	External user → trusted application.	Security boundary
Investigation workflow	Bound allowed reads and prepare evidence context.	Model suggestions → application decisions	Predictable investigation
Read-only tools	Access approved operational information	Application → business source systems	Controlled integrations
Source systems	Own metrics and deployment facts		Authoritative evidence
LLM/Analysis	Explain evidence, uncertainty and form hypotheses	Source records → untrusted model output	Flexible reasoning
Report validator	Check structure and evidence references	Untrusted result → user-visible report	Reject invalid or unsupported output

Design decision: These components do not need to be separate microservices. The API, workflow, tools and validator can initially live inside one Python application.

The source systems can be local synthetic JSON files in Stage 4.

CHAPTER

2. Ownership by component

This distinction is more important than the choice of framework.

Data or decision	Owner

Verified user identity	Authentication system / application
Access to operational records	Source-system authorization
Metric values	Monitoring system
Deployment history	Deployment system
Investigation execution	Investigation workflow
Generated explanation	LLM output, not authoritative evidence
Production remediation	Authorized operations service and human decision-maker

Component	Owns	Does not own
Investigation API	Request validation and user-context handling	The original business records
Workflow	Execution order and result state	User privileges
Read-only adapters	Calls into approved systems	Authorization grants generated by the model
Source systems	Metrics and deployment truth	LLM interpretation
Analysis component	Observations, hypotheses, recommendations	Operational authority
Report validator	Output shape and citation existence	Proof of semantic correctness

The assistant can summarize operational facts, but it cannot make itself the source of truth.

For example, if the LLM reports a deployment time that conflicts with the deployment system, the deployment record wins.

CHAPTER

3. Define the API and typed tool contracts

A contract describes the inputs and outputs a component expects.

Contracts let us develop and test components independently. They also reduce the chance that the model receives unexpected data.

For now, these are proposed application-level contracts, not MCP protocol definitions.

Contract A — Investigation API

REQUEST

```
{
  "incident_id": "INC-42",
  "service": "checkout-api",
  "question": "Why is Checkout API slow?"
}
```

The request does not supply a trusted `user_id`, role or tenant. The backend derives these from the authenticated session.

Contract B — Read-only tools

PYTHON TYPE CONTRACTS — ILLUSTRATIVE

```
from typing import TypedDict


class Principal(TypedDict):
    user_id: str
    tenant_id: str


class MetricsEvidence(TypedDict):
    evidence_id: str
    service: str
    p95_latency_ms: int
    baseline_p95_ms: int
    error_rate_percent: float


class DeploymentEvidence(TypedDict):
    evidence_id: str
    service: str
    version: str
    deployed_at: str


def get_metrics(
        service: str,
        principal: Principal
) -> MetricsEvidence:
    ...


def get_deployments(
        service: str,
        principal: Principal
) -> list[DeploymentEvidence]:
    ...
```

These describe **logical interfaces**, not a particular SDK or wire protocol. The mock implementation will add explicit errors, time bounds and validation.

The trusted `principal` is created by the application, never generated by the LLM.

Example tool outputs

`get_metrics`

Read only

```
{
  "evidence_id": "METRIC-07",
  "service": "checkout-api",
  "p95_latency_ms": 1800,
  "baseline_p95_ms": 240,
  "error_rate_percent": 3.1
}
```

`get_deployments`

Read only

```
[
  {
    "evidence_id": "CHANGE-03",
    "service": "checkout-api",
    "version": "v2.7.4",
    "deployed_at": "09:55"
  }
]
```

All records are fictional. Production contracts will also need complete timestamps, source provenance and explicit error types.

CHAPTER

4. Happy-path request sequence

We now walk through one successful investigation.

Step	Component	Operation
1	Engineer	Sends investigation request for INC-42
2	Investigation API	Authenticates and verifies incident access
3	Workflow	Calls `get_metrics` with the verified identity
4	Monitoring system	Authorizes access and returns `METRIC-07`
5	Workflow	Calls `get_deployments`
6	Deployment system	Authorizes access and returns `CHANGE-03`
7	Workflow	Builds a context containing only relevant evidence
8	LLM	Produces observations, hypotheses and next steps
9	Validator	Checks required fields and cited evidence IDs
10	Engineer	Receives the report for review

Contract C — Investigation result

```
{
  "run_id": "RUN-42",
  "incident_id": "INC-42",
  "status": "needs_review",
  "observations": [
    {
      "text": "Checkout API p95 latency is 1800 ms.",
      "evidence_ids": [
        "METRIC-07"
      ]
    },
    {
      "text": "Version v2.7.4 deployed at 09:55.",
      "evidence_ids": [
```

```
        "CHANGE-03"
      ]
    }
  ],
  "hypothesis": {
    "text": "Recent deployment may be related.",
    "evidence_ids": [
      "METRIC-07",
      "CHANGE-03"
    ]
  },
  "next_step": "Investigate latency by deployment version.",
  "action_executed": false
}
```

Here, `needs_review` means the investigation has findings for human review. It does not mean the incident has been resolved or that an operational change is authorized.

The validator can check that `METRIC-07` and `CHANGE-03` really exist in the retrieved evidence. It cannot, by itself, prove that the hypothesis is correct.

CHAPTER

5. Trust boundaries and authorization

There are three important boundaries.

Boundary	What must happen
User → Investigation API	Authenticate the caller and verify incident access
Workflow → Source systems	Enforce permitted tool operations and source-level authorization
Retrieved evidence → LLM	Treat results as untrusted data and prevent them from becoming instructions

Example: The engineer may be authorized to inspect Checkout API but not the Payroll API.

Even if the LLM requests `get_metrics("payroll-api")`, the application and monitoring service must deny the unauthorized request.

A prompt saying "only access Checkout API" is not sufficient security.

CHAPTER

6. Failure path — monitoring data unavailable

Suppose the monitoring system times out while the deployment tool succeeds.

```
+----------------------------+
| Investigation starts       |
+----------------------------+
              |
              v
+----------------------------+
| get_metrics                |
| Result: TIMEOUT            |
```

```
+----------------------------+
              |
              v
+----------------------------+
| get_deployments            |
| Result: CHANGE-03          |
+----------------------------+
              |
              v
+----------------------------+
| Evidence incomplete        |
| Cannot verify latency      |
+----------------------------+
              |
              v
+----------------------------+
| Return partial findings    |
| No root-cause conclusion   |
+----------------------------+
```

Expected result:

```
{
  "run_id": "RUN-42",
  "status": "insufficient_evidence",
  "available_evidence": [
    "CHANGE-03"
  ],
  "missing_evidence": [
    "metrics"
  ],
  "recommendation": "Retry metrics retrieval or investigate manually.",
  "action_executed": false
}
```

This is a deliberate reliability choice: partial evidence must not become a confident conclusion.

If **authorization** fails, the workflow must stop **before** protected data reaches the model. The API must return an access-denied result and must not disclose incident evidence to the model.

CHAPTER

7. Baseline execution limits

Keep execution bounded: Our first version has a proposed execution budget:

Control	Initial rule
Tool calls	One metrics read and one deployment-history read
Model calls	One analysis call
Production write tools	None
Source failure	Explicit incomplete-evidence result
Invalid citations	Reject or regenerate within a defined budget

Remediation	Human review, outside the initial execution path

These are design choices for the local baseline, not enterprise service limits.

We also do not need autonomous tool selection yet. A fixed workflow is more predictable because we already know which two records to retrieve.

Later, if the assistant needs to decide whether to inspect logs, database connections or deployment changes based on intermediate evidence, we can replace the fixed sequence with a bounded adaptive agent.

CHAPTER

8. What we will implement in Stage 4

The local mock will use this small structure:

```
it-ops-assistant/
|
+-- app.py
|   Investigation entry point
|
+-- tools.py
|   Read-only mock operations
|
+-- investigator.py
|   Workflow and evidence analysis
|
+-- data/
|   +-- metrics.json
|   +-- deployments.json
|
+-- tests/
    +-- test_investigation.py
```

We will create synthetic source records, wire the two tools to the workflow, and display the investigation result. We will also test unauthorized access and missing evidence.

No real production credentials or infrastructure will be needed.

CHAPTER

Stage 3 summary

We established the smallest useful design:

Engineer → Authorized API → Bounded workflow → Two read-only tools → Evidence → LLM → Validated report

The important architectural decisions are:

- Source systems own operational truth and enforce data permissions.
- Application code controls tool execution.
- The LLM produces explanations and hypotheses, not authorization decisions.
- Reports must distinguish facts, uncertainty and recommendations.
- Missing evidence produces an honest incomplete result.

- No production changes are permitted.

Next — Stage 4: Build a Working Local Mock

We will implement this design in Python, using synthetic metrics and deployment records. We will make the request and tool outputs visible and verify the behavior with software tests.

Continue to Stage 4